Outcomes

- N-Queen Problem.
- Genetic Algorithm implementation for N-Queen Problem.

---

1. Genetic Algorithm:

Genetic Algorithm is an optimization technique inspired by **natural selection**, where the best solutions evolve over time.
It works by repeatedly improving a population of solutions using **selection, crossover, and mutation**.

Biological Concepts (Steps of Genetic Algorithm):

Initial Population
Fitness Function
Selection
Crossover
Mutation

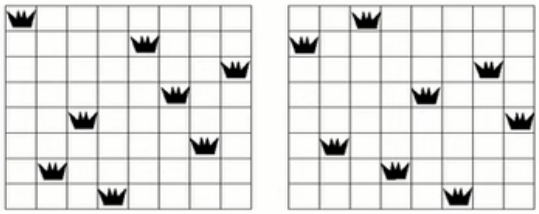
1. Population:

Biology

- Collection of individuals.

Algorithm

- Collection of states.



2. Fitness Function (Objective Function):

Biology

- More healthy, less prone to diseases.

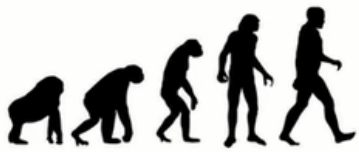
Algorithm

- Closest to the final solution.

3. Selection (based on the concept of "Survival of the fittest"):

Biology

- Selecting species that are the most biologically fit.



Algorithm

- Selecting states that are closest to the solution (Fittest).

4. Crossover:

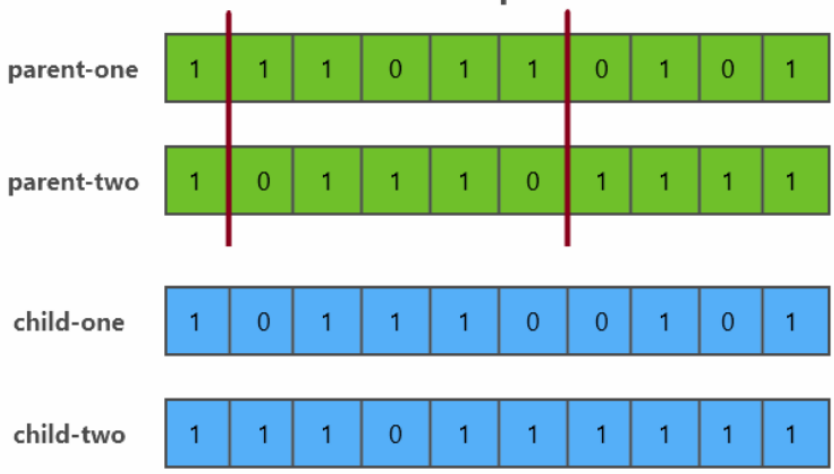
Biology

- Mating or Reproducing

Algorithm

- Interchanging values between selected states.

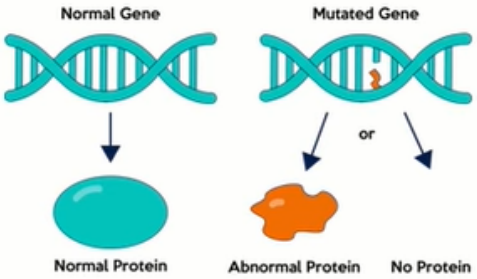
One Bit Addaptive Crossover



5. Mutation:

Biology

- Change or variation.



Algorithm

- Alteration.

Before Mutation

A5	1	1	1	0	0	0

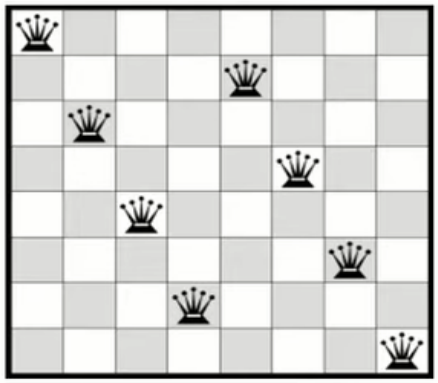
After Mutation

A5	1	1	0	1	1	0

2. N-Queen Problem

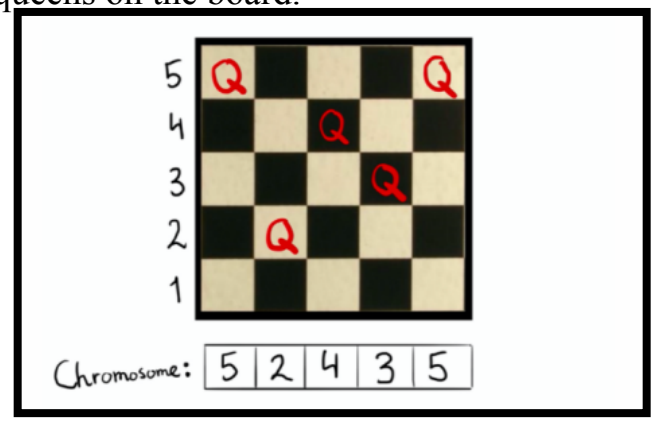
Objective:

- Arrange 8 queens on a standard chess board in such a way that no queen attacks each other.



6. Initial population

Initial population means a set of random possible solutions (chromosomes). Each chromosome represents one way of placing queens on the board.



In the initialization process, we need to arrange a random population of chromosomes (potential solutions) created. Here is the initial population, I took 4 chromosomes, each of which has a length 5. They are

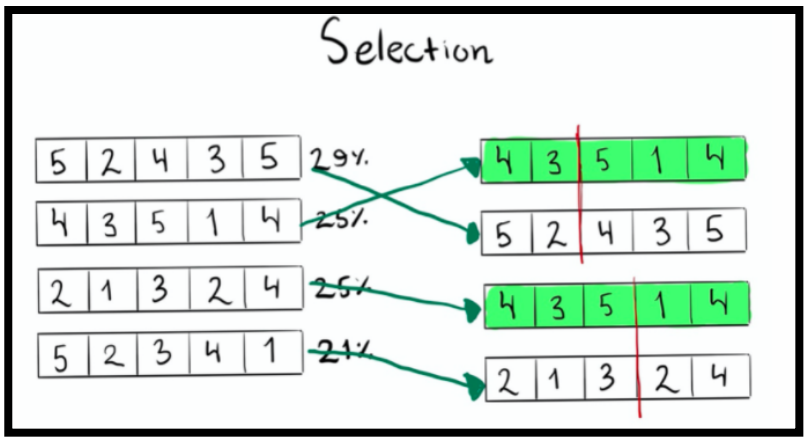
[5 2 4 3 5]
[4 3 5 1 4]
[2 1 3 2 4]
[5 2 3 4 1]

7. Fitness function

Fitness function measures how good a solution is. In N-Queen, it counts number of attacking queens.

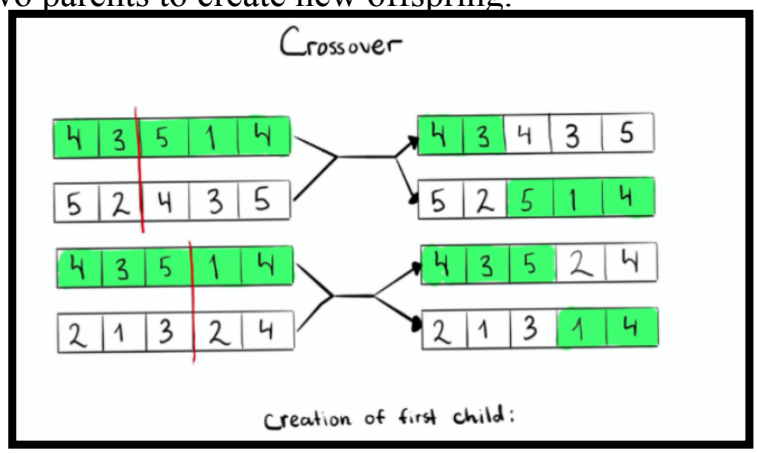
8. Selection

Selection chooses the best chromosomes based on fitness. Better solutions have higher probability of being selected.



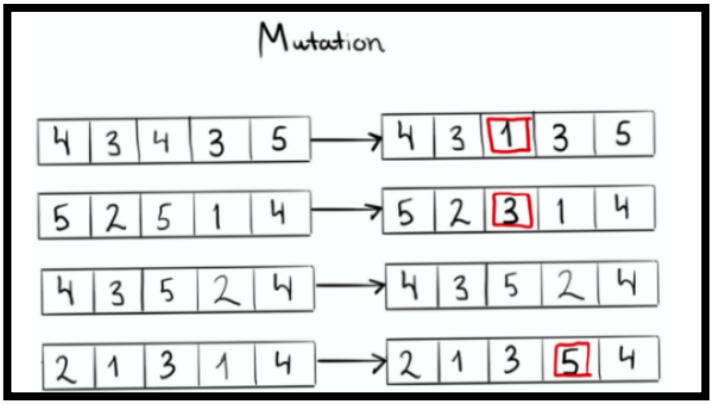
9. Crossover

Crossover combines two parents to create new offspring.

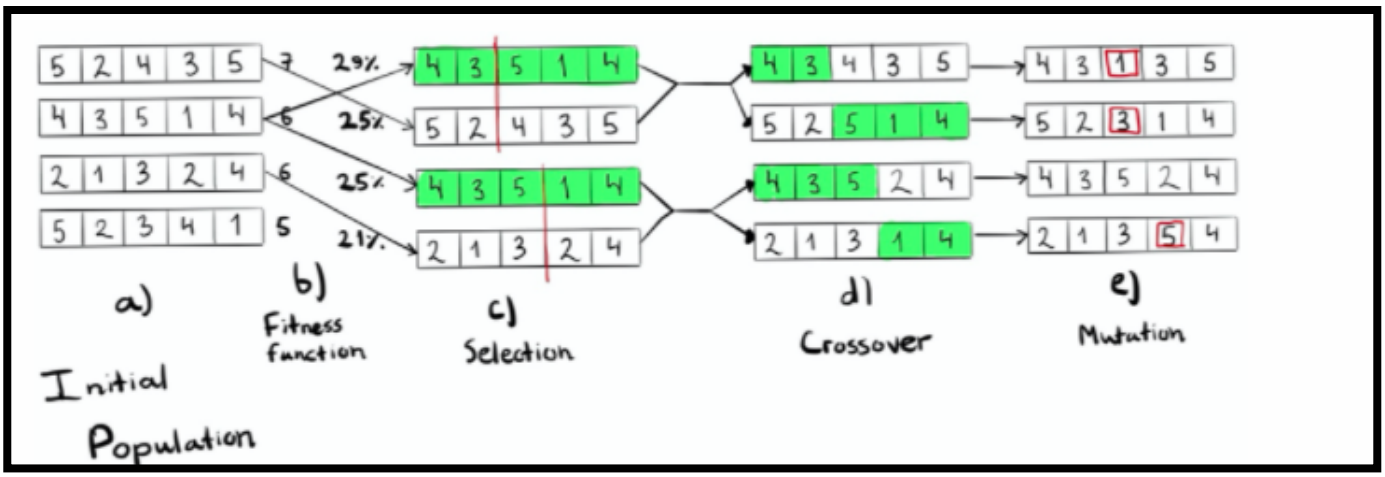


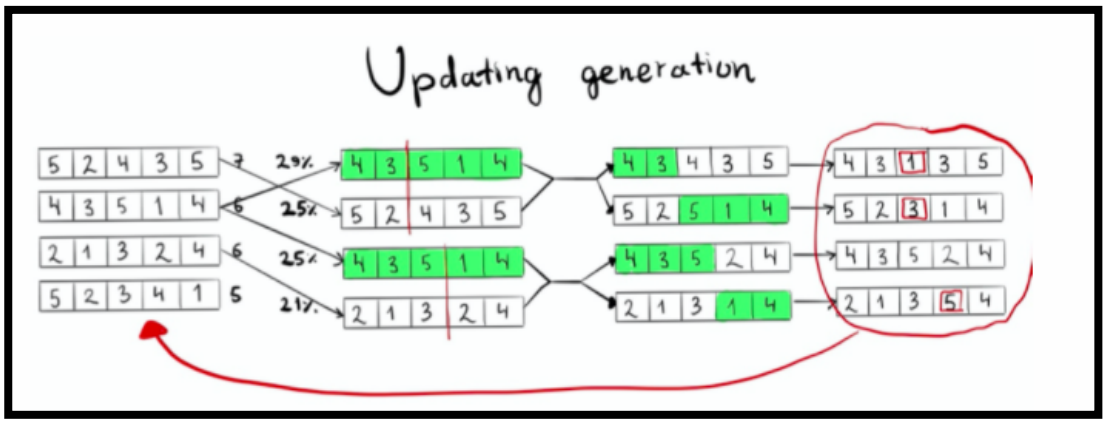
10. Mutation

Mutation randomly changes a gene to maintain diversity. It helps avoid getting stuck in bad solutions.



So until this, the genetic algorithm to solve the 5-Queen algorithm will look like the following:





➔ In the next step, we need to **update the generation.** New chromosomes will update the population but the population number will not change. So the following chromosomes will be our new population:

[4 3 1 3 5]
[5 2 3 1 4]
[4 3 5 2 4]
[2 1 3 5 4]

Implementation:

```
import copy
import random
from math import exp
import numpy as np

def make_board():
  rows, cols = (board_size, board_size)
  board=[]

  for i in range(rows):
    col=[]
    for j in range(cols):
      col.append("*")
    board.append(col)
  return board

def placeQueens(board):
  i=0
  while(i<board_size):
    row = random.randint(0, board_size-1)
    if board[row][i]!="Q":
      board[row][i]="Q"
      i+=1
  return board

def print_board(board):
  for i in range(board_size):
    for j in range(board_size):
      print(board[i][j], end=' ')
    print()

def getQueens(board, board_size):
  queen_positions=[]
  for i in range(board_size):
    for j in range(board_size):
      if board[i][j]=="Q":
        temp=i,j
        queen_positions.append(temp)
  return queen_positions


def make_chromosome(board):
  temp=[]
  for i in range(board_size):
    for j in range(board_size):
      if board[j][i]=="Q":
        temp.append(j)
```

```
  return temp


def generate_population(size):
  populat =[]
  for i in range(size):
    b = make_board()
    temp =placeQueens(b)
    temp2 = make_chromosome(temp)
    if temp2 not in populat:
      populat.append(temp2)
  return populat

import numpy.random as npr
def selectOne(fit):
  max=sum([c for c in fit])
  selection_probs=[c/max for c in fit]
  return fit.index(fit[npr.choice(len(fit), p=selection_probs)])

def selection(population, fit):
  new_p1 = copy.deepcopy(population)
  new_f1 = copy.deepcopy(fit)

  index1 = selectOne(new_p1)
  c1, f1=new_p1[index1], new_f1[index1]

  index2 = selectOne(new_p1)
  c2, f2=new_p1[index2], new_f1[index2]

  return c1, f1, c2, f2

def crossover(c1, c2):
  ra=np.random.randint(0,4)
  c3=c1[:ra]+c2[ra:]
  c4=c2[:ra]+c1[ra:]
  return c3, c4
def mutation(of_Spring, mutRate):
  pass

def Evaluate(of_Sp, Of_Fit, parent, p_fit):
  pass

def objecive_function(board):
  queen_pos=getQueens(board, board_size)
  h_hit=0
  d_hit=0
  for i in range(0, len(queen_pos)):
    for j in range(i, len(queen_pos)):
      if i != j:
        if queen_pos[i][0]==queen_pos[j][0]:
```

```
            h_hit+=1
        if abs(queen_pos[i][0] - queen_pos[j][0]) == abs(queen_pos[i][1] -
queen_pos[j][1]):
            print("queen_pos[i][0] - queen_pos[j][0]", queen_pos[i][0] , queen_pos[j][0])
            print("queen_pos[i][1] - queen_pos[j][1]", queen_pos[i][1], queen_pos[j][1])
            d_hit+=1
  return h_hit+d_hit

def main():
  global board_size
  board_size = int(input("Enter the board size: "))

  g = generate_population(board_size)
  print(g)
  board=make_board()
  print_board(board)
  placeQueens(board)
  print(getQueens(board, board_size))

  print_board(board)
  print("\nObjective Function", objecive_function(board))

if __name__=="__main__":
  main()
```

Outputs

1.

```
Enter the board size: 5
[[3, 2, 2, 2, 3], [1, 1, 0, 4, 4], [2, 0, 2, 3, 1], [0, 4, 2, 3, 1], [4, 1, 0, 1, 4]]
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
[(0, 2), (1, 0), (2, 1), (3, 3), (4, 4)]
* * Q * *
Q * * * *
* Q * * *
* * * Q *
* * * * Q
queen_pos[i][0] - queen_pos[j][0] 1 2
queen_pos[i][1] - queen_pos[j][1] 0 1
queen_pos[i][0] - queen_pos[j][0] 3 4
queen_pos[i][1] - queen_pos[j][1] 3 4

Objective Function 2
```

2.

```
Enter the board size:5
[[1]]
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
[(0, 0), (0, 4), (1, 2), (2, 1), (3, 3)]
Q * * * Q
* * Q * *
* Q * * *
* * * Q *
* * * * *
queens_pos[i][0] - queens_pos[j][0] 0 3
queens_pos[i][1] - queens_pos[j][1] 0 3
queens_pos[i][0] - queens_pos[j][0] 1 2
queens_pos[i][1] - queens_pos[j][1] 2 1

 Objective Function 3
```

Lab Tasks:

1. Write Python code for N-Queen problem Implementation using Genetic Algorithm.

2. Understand the working of the code step-by-step and analyze + explain how each function behaves.

   ➔ For every function, write:
   - What is the **purpose** of the function?
   - What are its **inputs (parameters)**?
   - What does it **return**?

3. **Dry Run the Code**

   - Take a small example (e.g., board_size = 5 or 8)
   - Manually trace the execution step-by-step

   Must show:

   - Initial empty board
   - Placement of queens
   - Generated chromosome
   - Queen positions
   - Objective function calculation (conflicts)

4. Briefly explain with the help of code that how Genetic Algorithm is helpful in solving N-Queen problem

Note: You must submit pdf containing all the explanations and Dry running of the code.

Lab 07: Genetic Algorithm & N-Queen Problem

Lab # 07 Marks distribution

	ER1	ER6	ER8
Task	3 points	3 points	4 points

Lab # 07 Rubric Evaluation Guideline:

#	Qualities & Criteria	0 < Poor <= 1	1 < Satisfactory <= 2	2 < Excellent <=3
ER1	**Task Completion**	Minimal or no program functionality was achieved.	Some tasks were completed, but the program has errors or incomplete functionalities.	All tasks were completed, and the program runs without errors.
#	**Qualities & Criteria**	**0 < Poor <= 1**	**1 < Satisfactory <= 2**	**2 < Excellent <=3**
ER6	**Program Output**	Output is inaccurate or poorly presented.	Output is mostly accurate but may lack labels, captions, or formatting.	Output is clear, accurate, and well presented with labels, captions, and proper formatting.
#	**Qualities & Criteria**	**0 < Poor <= 1**	**1.5 < Satisfactory <= 2.5**	**2.5 < Excellent <= 4**
ER8	**Question & Answer**	Answers some questions but not confidently or based on lab task knowledge.	Answers most questions confidently and based on lab task knowledge.	Answers all questions confidently and demonstrates a deep understanding of the given lab task.